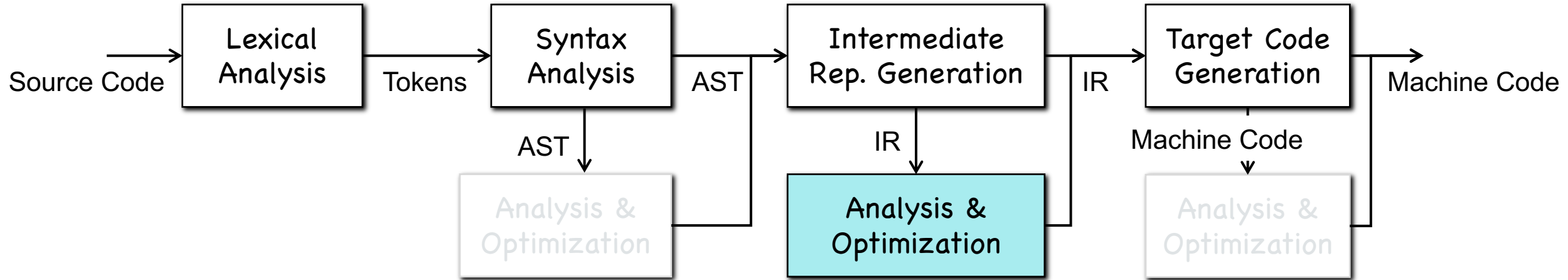


Chapter 9-2

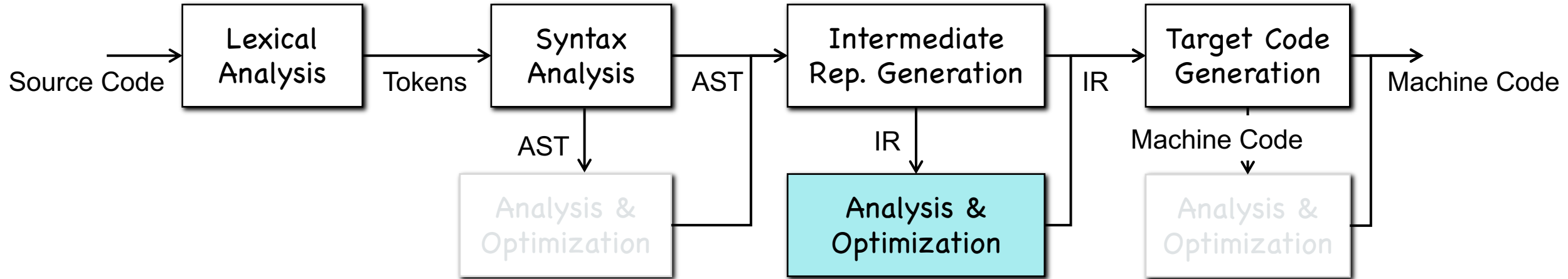
Symbolic Execution

Data Flow Analysis



- Data flow analysis is a form of static program analysis
 - Computes dataflow facts held at a program point
 - E.g., what definitions can reach the 2nd basic block?
 - **Follow the order of control flows (order of statements/blocks)**

Data Flow Analysis



- Data flow analysis is a form of static program analysis

- Computes dataflow facts held at a program point

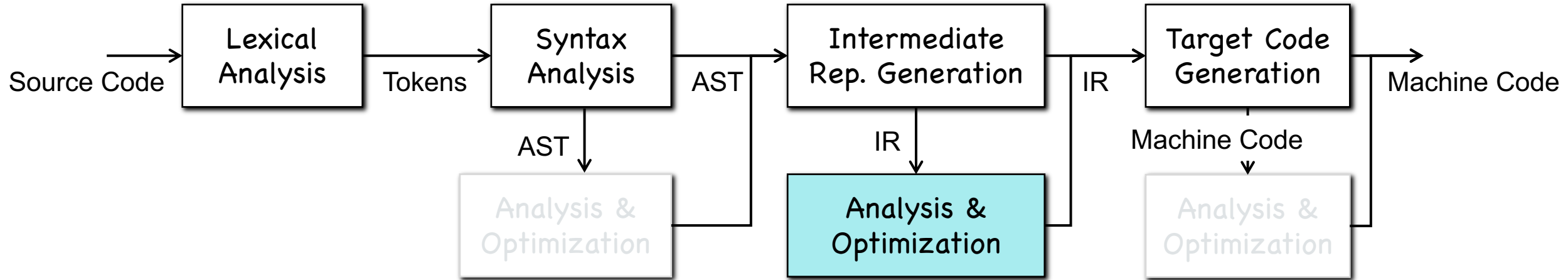
- E.g., what definitions are held at a block?

- Follow the order of definitions (statements/blocks)

Flow Sensitive!

Is a data flow fact held at a program point?

Data Flow Analysis



Is a data flow fact held **at a program point**?

Flow Insensitive

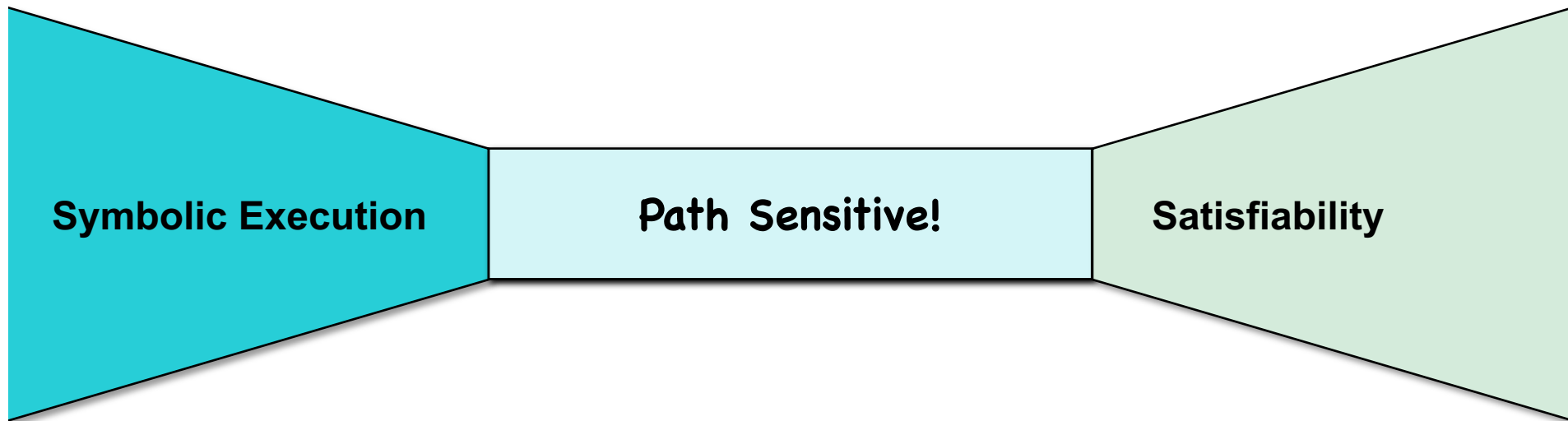
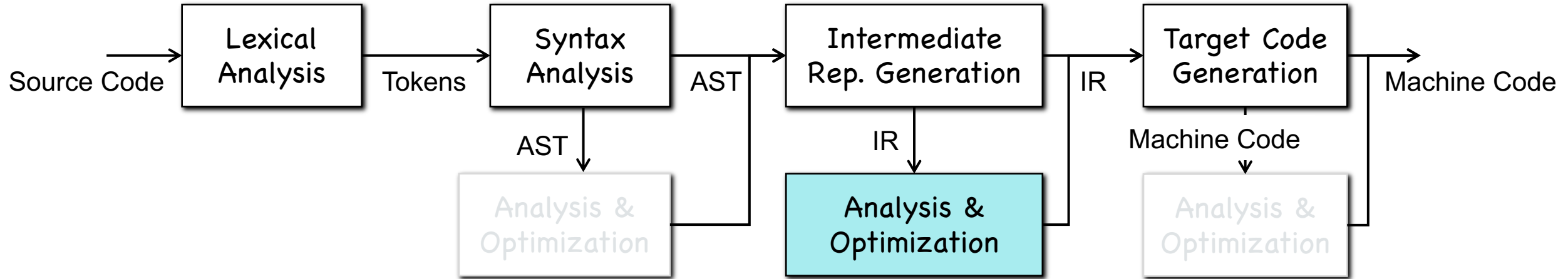
Flow Sensitive!

Advantage: High Precision
Disadvantage: Path Explosion

Path Sensitive

Is a data flow fact held **along a program path**?

Data Flow Analysis



PART I: Symbolic Execution

Symbolic Execution

- Symbolic execution is a classic path-sensitive analysis that enumerates and analyzes each path of a program
 - Bug finding;
 - Software testing;
 - ...



<https://github.com/klee/klee>

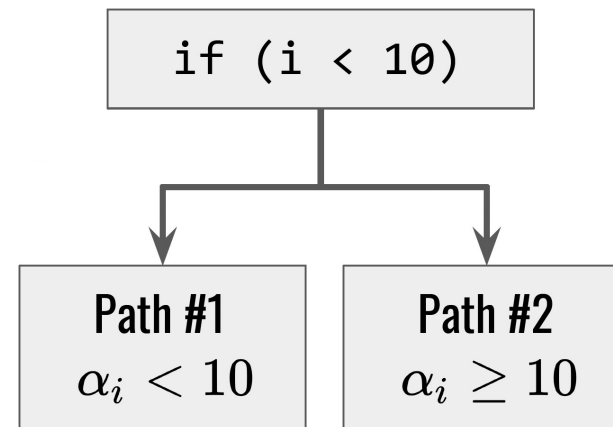


<https://clang-analyzer.llvm.org/>

```
clang -analyze -Xanalyzer -analyzer-checker=core.DivideZero hello.c
```

Symbolic Execution

- Each input variable is associated to a symbol, *e.g.*, **int i $\rightarrow$ α_i**
- Each program statement generate formulas over the input symbols, *e.g.*, **$i * 2 + 5 \rightarrow 2 * \alpha_i + 5$**
- Program path $\Rightarrow$ path constraint



Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert (x - y != 0);
9.  }

```

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = true$

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = \alpha_a = 0$
check: $\pi \wedge \neg(1 - 0 \neq 0) \Leftrightarrow false$ **ok**

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = true$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 4 \};$$

$$\pi = \alpha_a \neq 0$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 2(\alpha_a + \alpha_b) ; y \rightarrow 4 \};$$

$$\pi = \alpha_a \neq 0 \wedge \alpha_b = 0$$

check: $\pi \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$
satisfiable => error

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver
 - Satisfiable $\Rightarrow$ A possible solution
 - Unsatisfiable
 - Unknown (due to timeout)



<https://github.com/Z3Prover/z3>



<https://cvc5.github.io>

Limitations

- **Scalability**

- (1) Path Explosion
- (2) External Calls
- (3) Constraint Solving

- **Optimizations**

- (1) Path/State Merging
- (2) Induction for Loops
- (3) Concolic Execution
- (4) Speculative Execution
- (5)

Path (or State) Merging

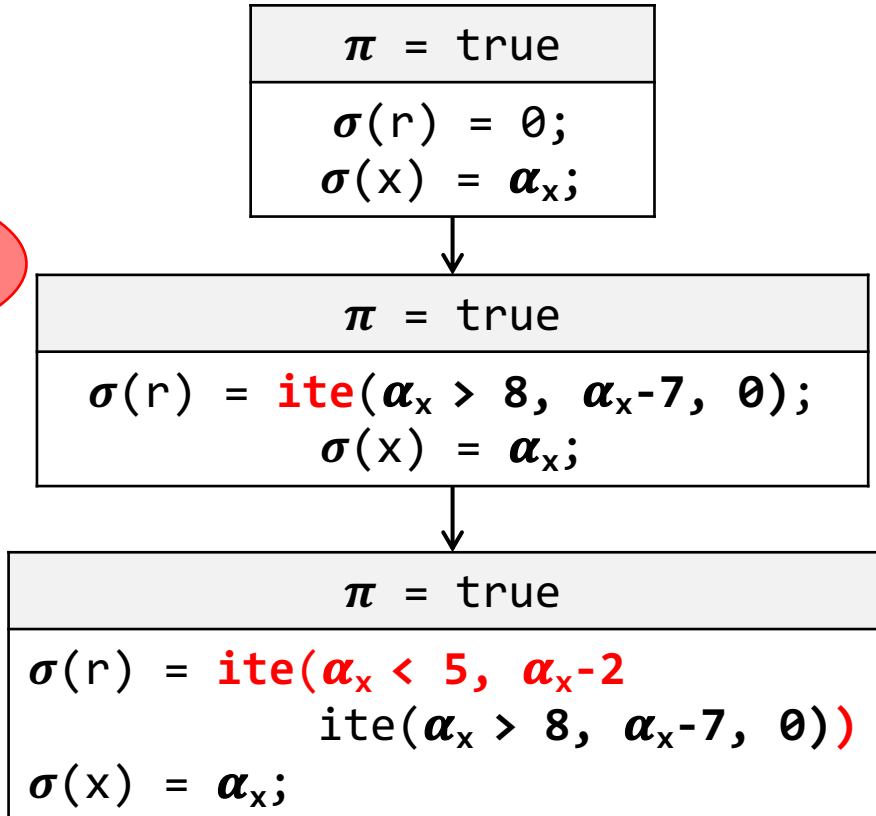
```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

Avoid repetitive
analysis!

More complicated
constraints!



Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.

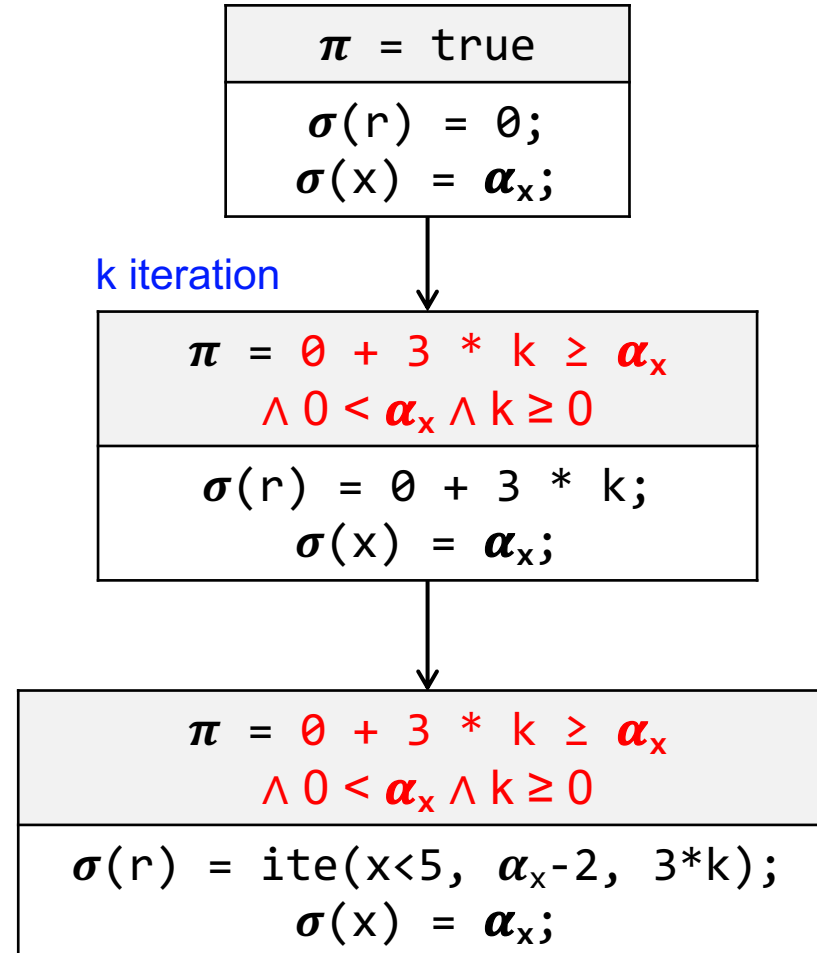
Induction for Loops

```

1  void foo(int x) {
2      int r = 0;
3
4      while (r < x)
5          r += 3;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }

```

Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.



Induction for Loops

```
1 void foo(int x) {  
2     int r = 0;  
3  
4     while (r < x)  
5         if (*) r += 3; else r *= 4;  
6  
7     if (x < 5)  
8         r = x - 2;  
9  
10    assert(r != 5);  
11 }
```

It does not
always work!

Concolic Execution

- Concrete + Symbolic Execution
- Perform symbolic execution at runtime

Concolic Execution

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (sin(x) < 0.5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

$\pi = \text{true}$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5$	
$\sigma(r) = \alpha_x - 2;$	$\mathcal{C}(r) = -1;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

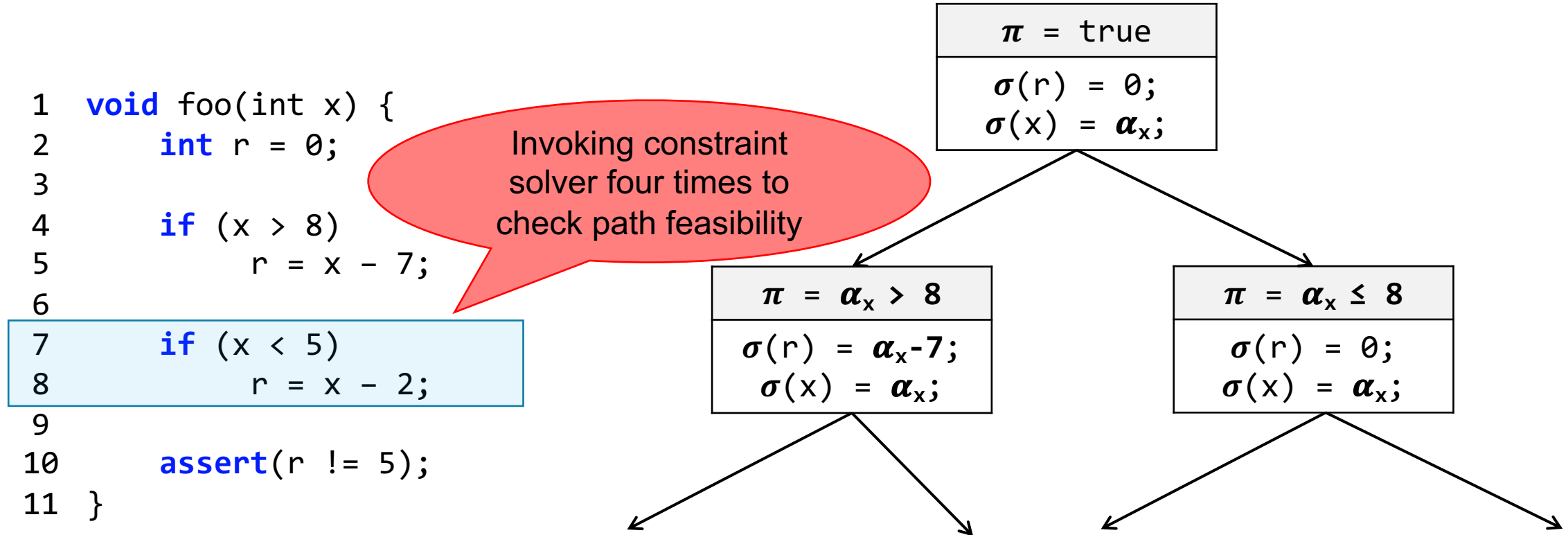
Advantages:

- (1) Handle external calls
- (2) Ease constraint solving

Check if $\alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5 \wedge \alpha_x - 2 = 5$ is satisfiable.

Replace $\sin(x)$ with the concrete value $\sin(1)$, i.e., 0.1

Speculative Execution



Speculative Symbolic Execution: Provide a way to guess path feasibility w/o constraint solving, thereby reducing the invocation of the constraint solver!

PART II: Satisfiability

Solving Path Constraints

- Propositional Logic
 - Satisfiability (SAT) --- DPLL and CDCL
-
- First Order Logic
 - Satisfiability Modulo Theories (SMT)
 - The Bit Vector Theory

Logic

- A logic is just a formal language (just like regular languages, etc.)
- (1) **Syntax**: What are the legal sentences of the language?
- (2) **Semantics**: What does a sentence mean?

Propositional Logic (PL)

- **Syntax of PL**, i.e., what are legal sentences of PL?
 - (1) **Truth Symbols (b)**: $\top$ and $\perp$ (true and false)
 - (2) **Propositional Variables (v)**: $p, q, r, \dots$
 - (3) **Literals (l)**: atom or its negation
 - (4) **Formulas (F)**: $l \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2$
- } Atoms

Propositional Logic (PL)

- What are legal sentences of PL?
 - $\perp$
 - $\top \wedge \perp$
 - $p \vee q$
 - $p \text{ and } q$
- How would you encode “If it is cold and rainy, then it is either warm or rainy” as a formula in PL?

Satisfiability and Validity

- A **solution** of a formula F in PL is a map I that maps each variable to either $\top$ or $\perp$
- A formula F in PL is **satisfiable**, iff there exists $I: I \models F$
- A formula F in PL is **valid**, iff for all $I: I \models F$

Theorem [Duality of Satisfiability and Validity]

For all formulae F in propositional logic, F is valid if and only if $\neg F$ is not satisfiable.

Normal Forms

- A normal form is a syntactic restriction on the shape of a formula
- A normal form structure the search space and simplify the decision procedure

Negation Normal Form (NNF)

- **Negation normal form** disallows implications and only allows variables to be negated
 - (1) **Truth Symbols (b)**: $\top$ and $\perp$ (true and false)
 - (2) **Propositional Variables (v)**: $p, q, r, \dots$
 - (3) **Literals (l)**: atom or its negation
 - (4) **Formulas (F)**: $l \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid \cancel{F_1 \Rightarrow F_2} \mid \cancel{F_1 \Leftrightarrow F_2}$

Negation Normal Form (NNF)

- **Negation normal form** disallows implications and only allows variables to be negated
- Which of these formulas are in NNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $\neg p \vee \neg q \vee p \vee \neg q$
 - $\neg(p \wedge q) \vee p \vee \neg q$

Theorem [Sufficiency of NNF]

For all formulae F in propositional logic, there exists an equivalent formula in negation normal form.

Disjunctive Normal Form (DNF)

- Formulas in DNF are disjunctions of conjunctive literals

$$\bigvee_i \bigwedge_j l_{ij}$$

- Which of these formulas are in DNF?

- $(p \wedge q) \Rightarrow (p \vee \neg q)$
- $\neg p \wedge \neg q \vee (p \wedge \neg q)$
- $\neg(p \wedge q) \vee p \vee \neg q$
- $\neg p \vee \neg q \vee p \vee \neg q$



Theorem [Sufficiency of DNF]

For all formulae F in PL, there exists an equivalent formula in DNF.

Conjunctive Normal Form (CNF)

- Formulas in CNF are conjunctions of disjunctive literals

$$\bigwedge_i \bigvee_j l_{ij}$$

- Which of these formulas are in CNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $(\neg p \vee \neg q) \wedge (p \vee \neg q)$
 - $\neg p \vee \neg q \vee p \vee \neg q$

Theorem [Sufficiency of CNF]

For all formulae F in PL, there exists an equivalent formula in CNF.

Equisatisfiability

- Two formulas F_1 and F_2 are equisatisfiable iff:
$$F_1 \text{ is satisfiable} \Leftrightarrow F_2 \text{ is satisfiable}$$
- Equisatisfiability $\neq$ Equivalence

Theorem [Efficient CNF Encoding]

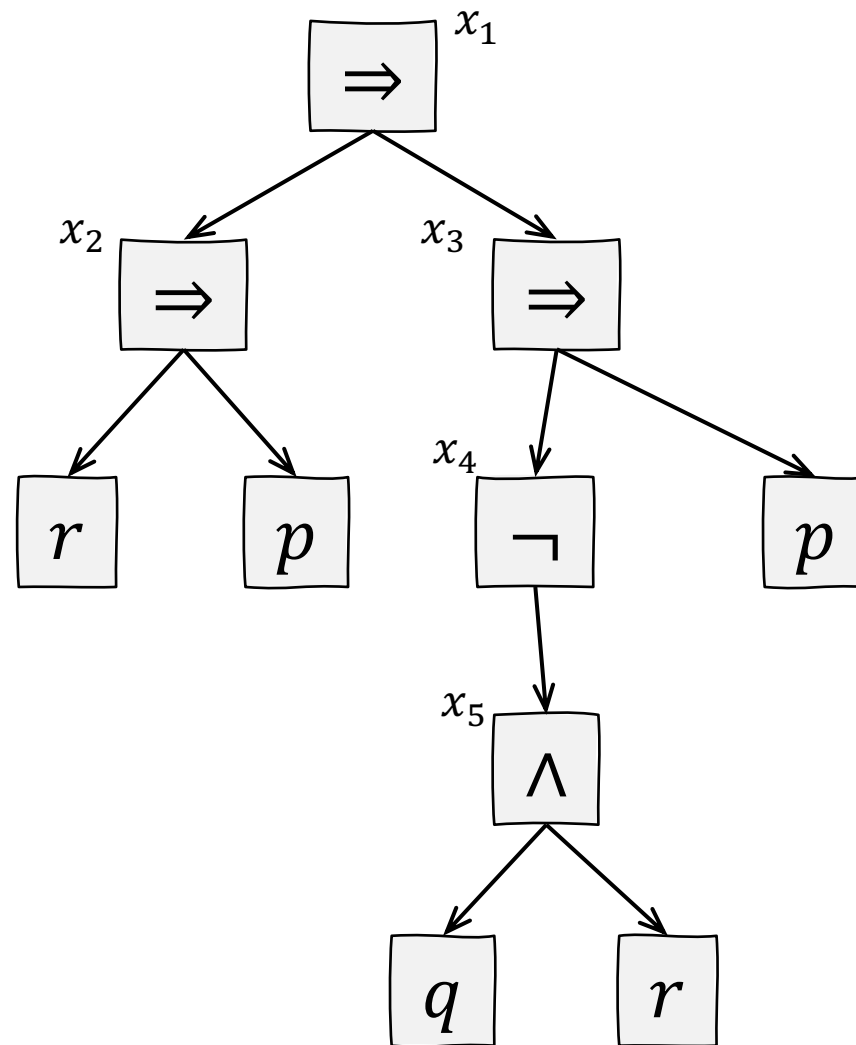
For all formulae F in PL, there exists an equisatisfiable formula in CNF that is at most a constant factor larger than F .

Tseitin Transformation

$$\bullet (r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$$

Diagram illustrating the Tseitin Transformation of the logical formula $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$. The formula is annotated with subexpressions and their corresponding indices:

- x_2 is associated with $r \Rightarrow p$.
- x_3 is associated with $\neg(q \wedge r) \Rightarrow p$.
- x_5 (highlighted in blue) is associated with $\neg(q \wedge r)$.
- x_4 is associated with $\neg(q \wedge r) \Rightarrow p$.
- x_1 is associated with the entire formula $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$.



Tseitin Transformation

$$\bullet (r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$$

Diagram illustrating the Tseitin Transformation with variable assignments:

- x_2 is assigned to r .
- x_3 is assigned to p .
- x_4 is assigned to $\neg(q \wedge r)$.
- x_5 is assigned to q .
- x_1 is assigned to the entire expression $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$.

$$\begin{aligned} & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\ & \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\ & \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p) \\ & \wedge x_4 \Leftrightarrow \neg x_5 \\ & \wedge x_5 \Leftrightarrow q \wedge r \end{aligned}$$

$$x_4 \Rightarrow \neg x_5 \wedge \neg x_5 \Rightarrow x_4$$

$$(\neg x_4 \vee \neg x_5) \wedge (x_5 \vee x_4)$$

Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \Rightarrow \underbrace{(\neg(q \wedge r) \Rightarrow p)}_{x_4} \end{array} \\
 \underbrace{\hspace{10em}}_{x_1}
 \end{array}
 \qquad
 \begin{array}{c}
 \overbrace{(\neg(q \wedge r) \Rightarrow p)}^{x_3} \\
 \underbrace{\hspace{2em}}_{x_5}
 \end{array}$$

$$\begin{aligned}
 & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\
 & \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\
 & \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p) \\
 & \wedge x_4 \Leftrightarrow \neg x_5 \\
 & \wedge x_5 \Leftrightarrow q \wedge r
 \end{aligned}$$

Theorem [Efficient CNF Encoding]

For all formulae F in PL, there exists an equisatisfiable formula in CNF that is at most a constant factor larger than F .

The SAT Problem

- Deciding satisfiability of a formula F in PL
- Find a solution such that we can rewrite F to be true

Recap: A **solution** of a formula F in PL is a map I that maps each variable to either $\top$ or $\perp$

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$
- **Goal:** Find a solution to satisfy the input formula
- Repeat the following steps:
 - (1) Choose a variable, p , assign true or false to the variable
 - (2) Propagate the value of p

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```

bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
  
```

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$
- $(p \vee q) \wedge (p \vee \neg q) \wedge \neg p$
- $p \wedge \neg p$ **False! Unsatisfiable!**

Theorem [Resolution]

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee \mathbf{c}) \wedge (b_0 \vee b_1 \vee \cdots \vee b_m \vee \neg \mathbf{c})$$

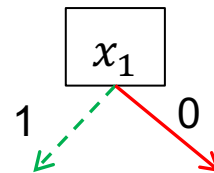
can be simplified into

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee b_0 \vee b_1 \vee \cdots \vee b_m)$$

CDCL

- Conflict-Driven Clause Learning
- Make DPLL faster by learning from mistakes

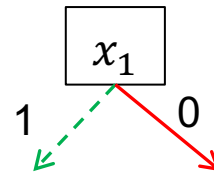
CDCL



$$x_1 = 0$$

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

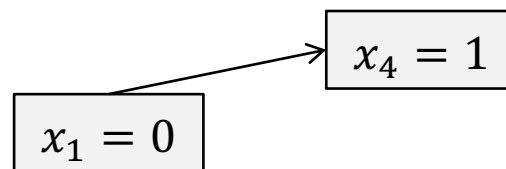
CDCL



$$x_1 = 0; x_4 = 1$$

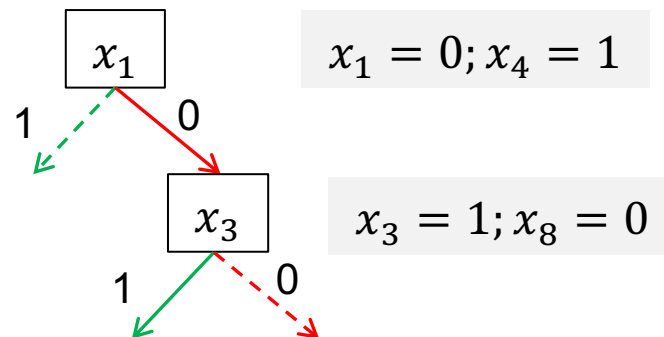
- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

Implication Graph

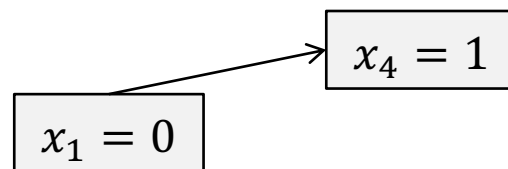


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

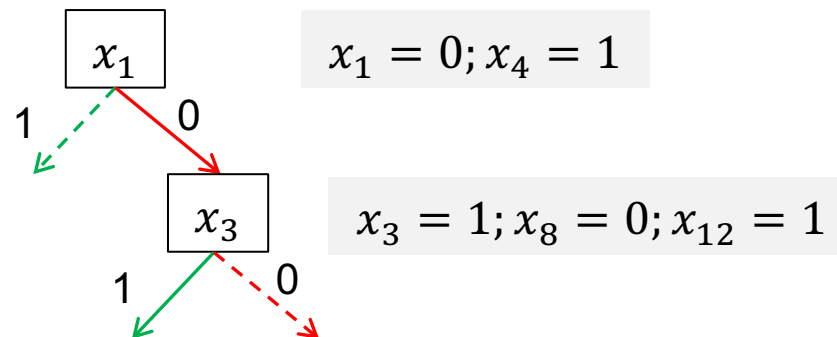


Implication Graph

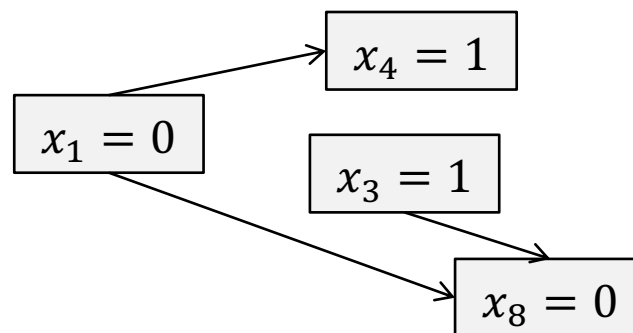


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

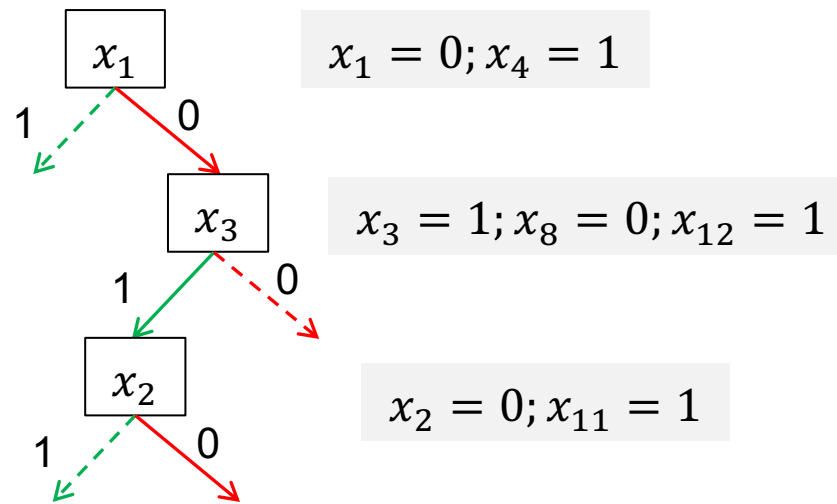


Implication Graph

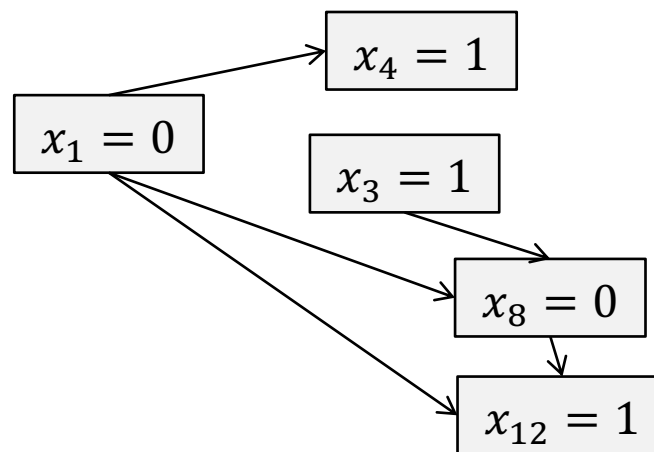


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

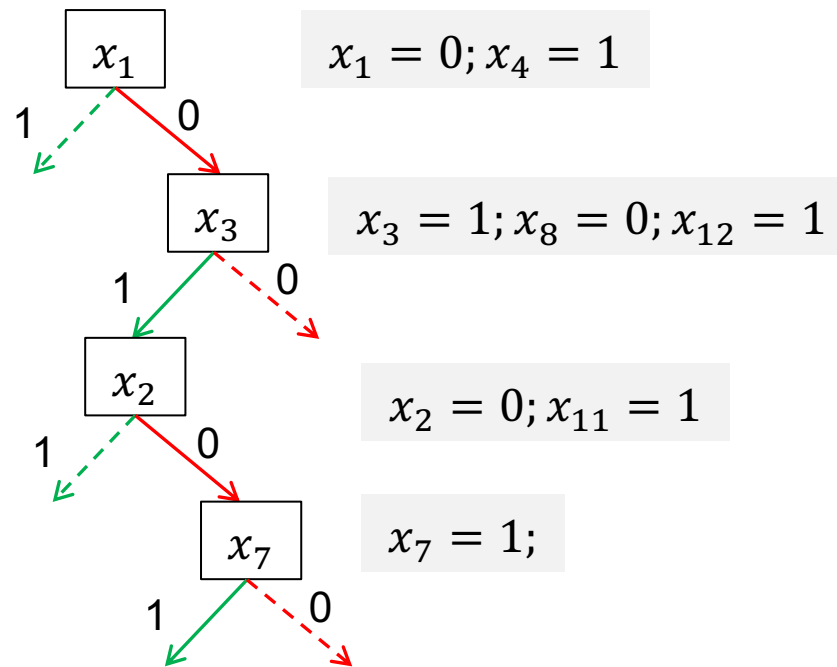


Implication Graph

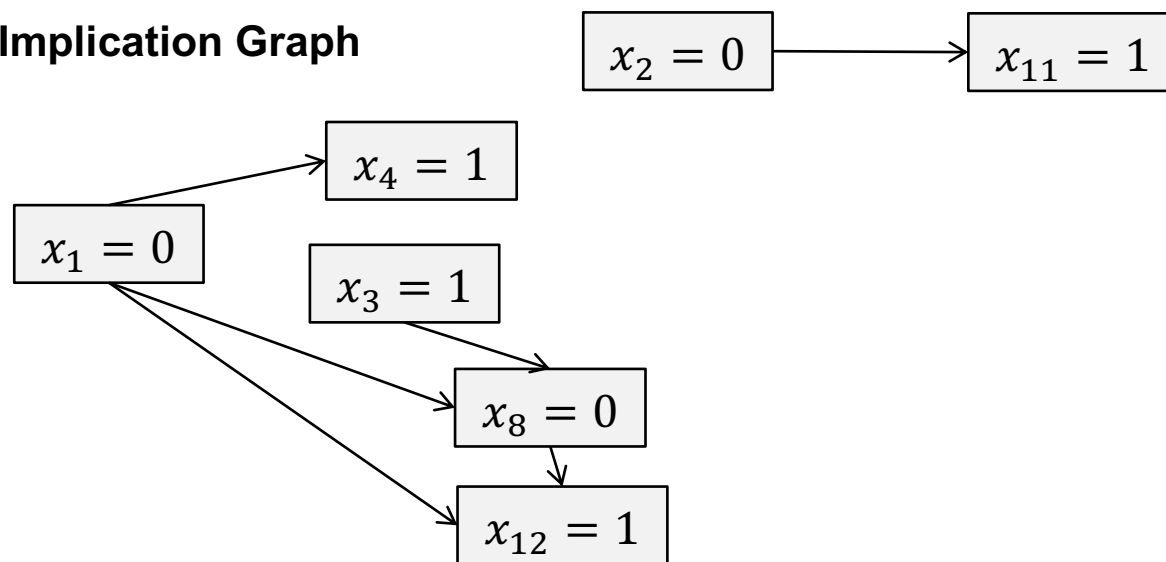


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

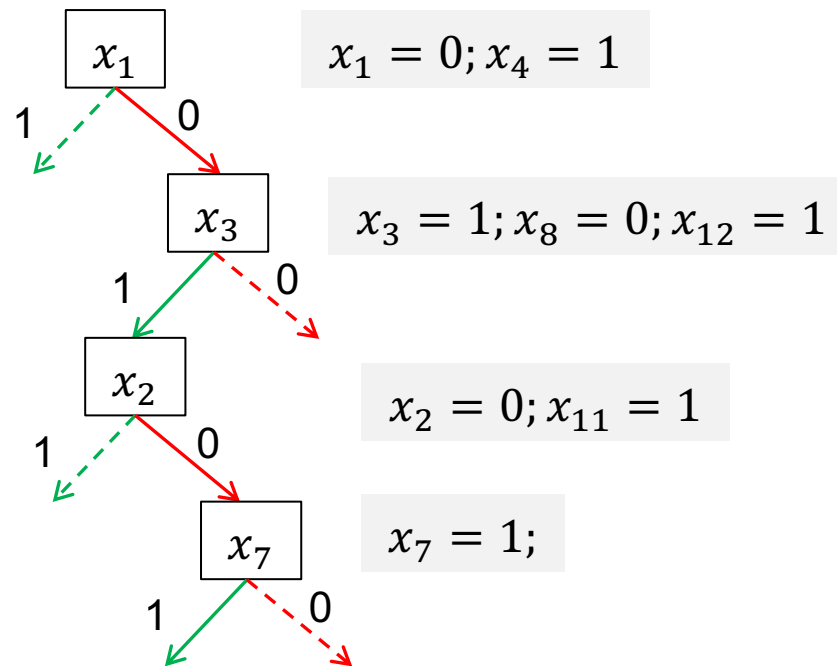


Implication Graph

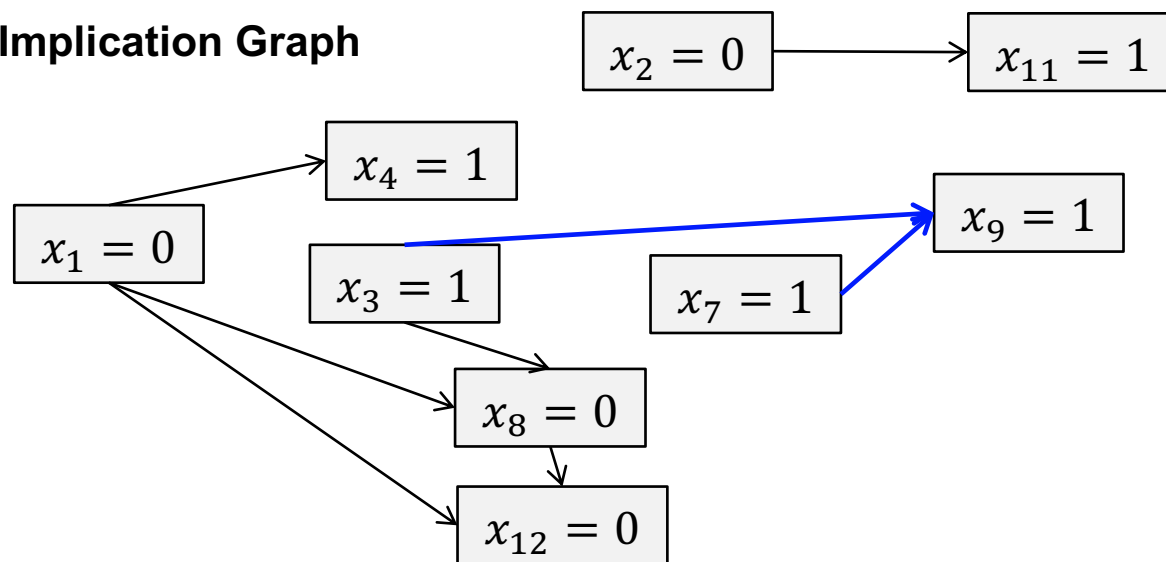


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

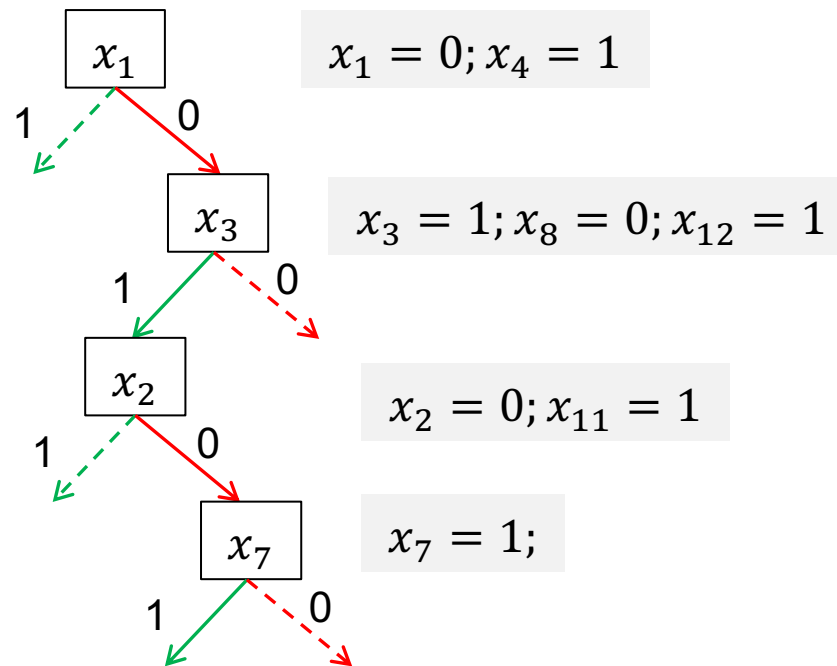


Implication Graph

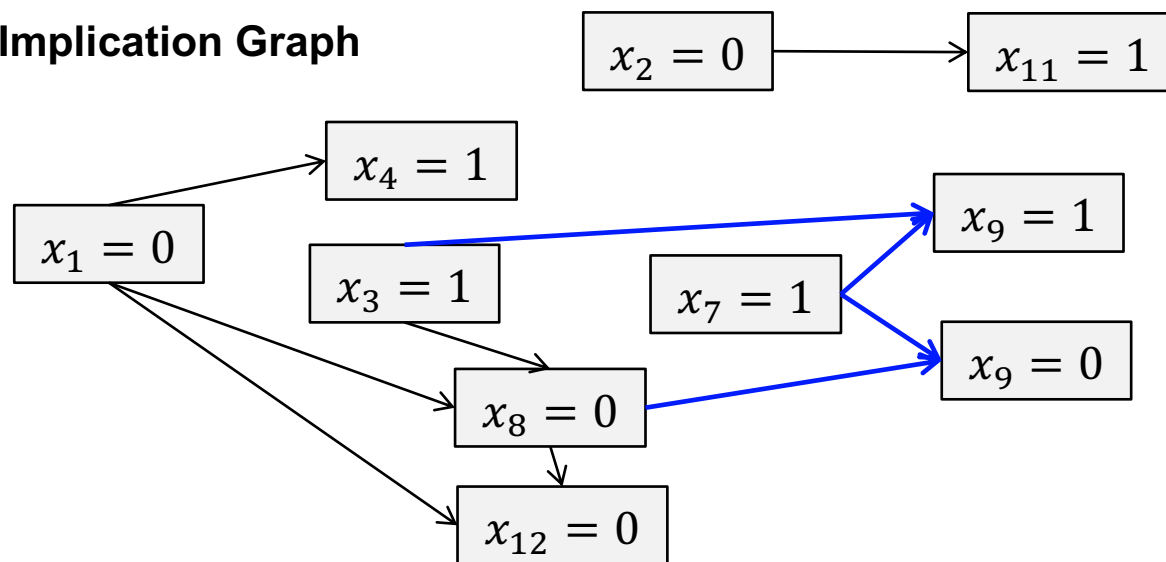


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

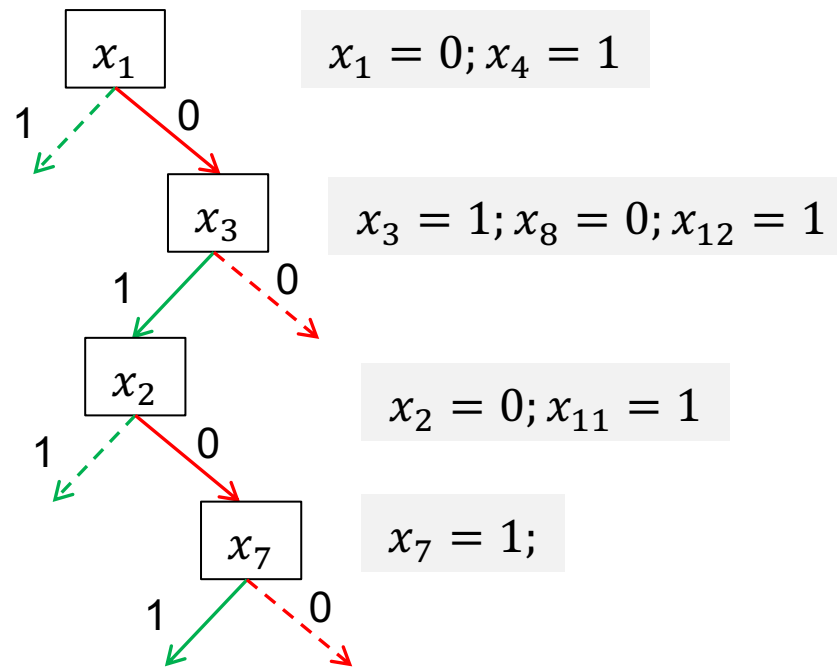


Implication Graph

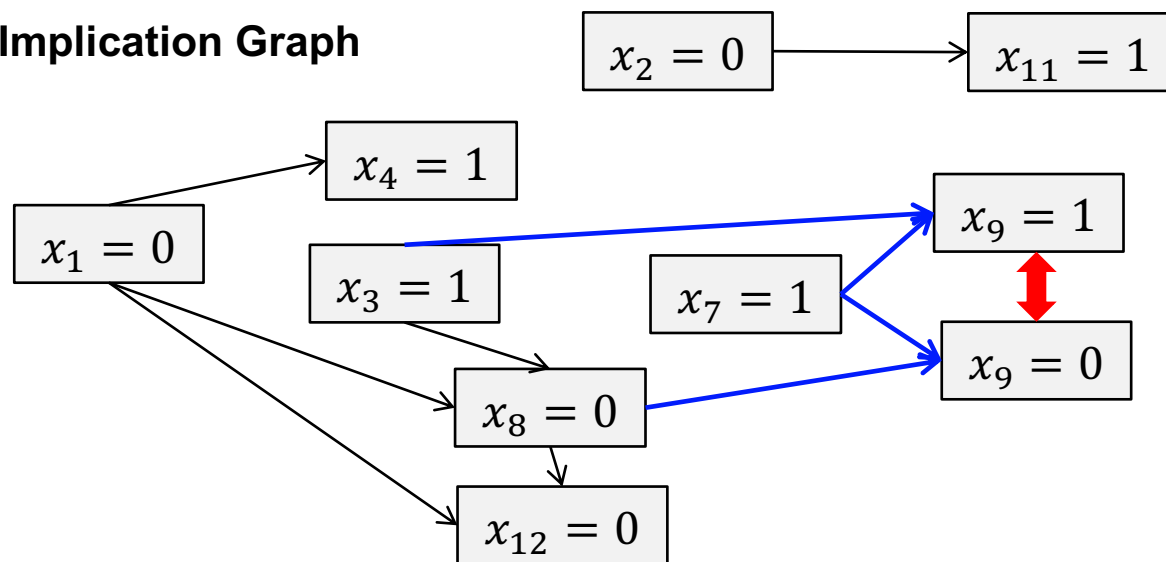


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

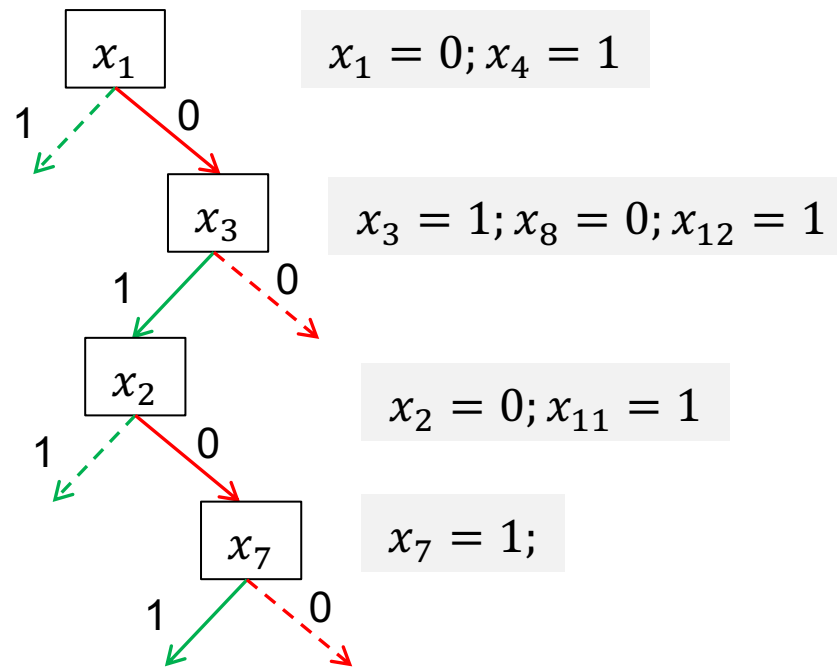


Implication Graph

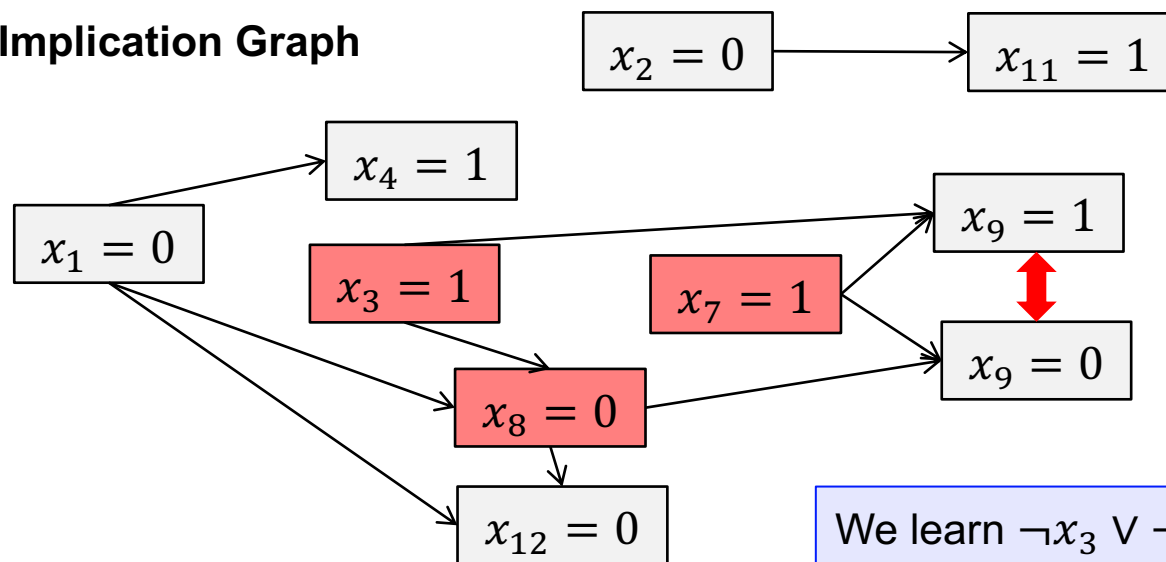


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$



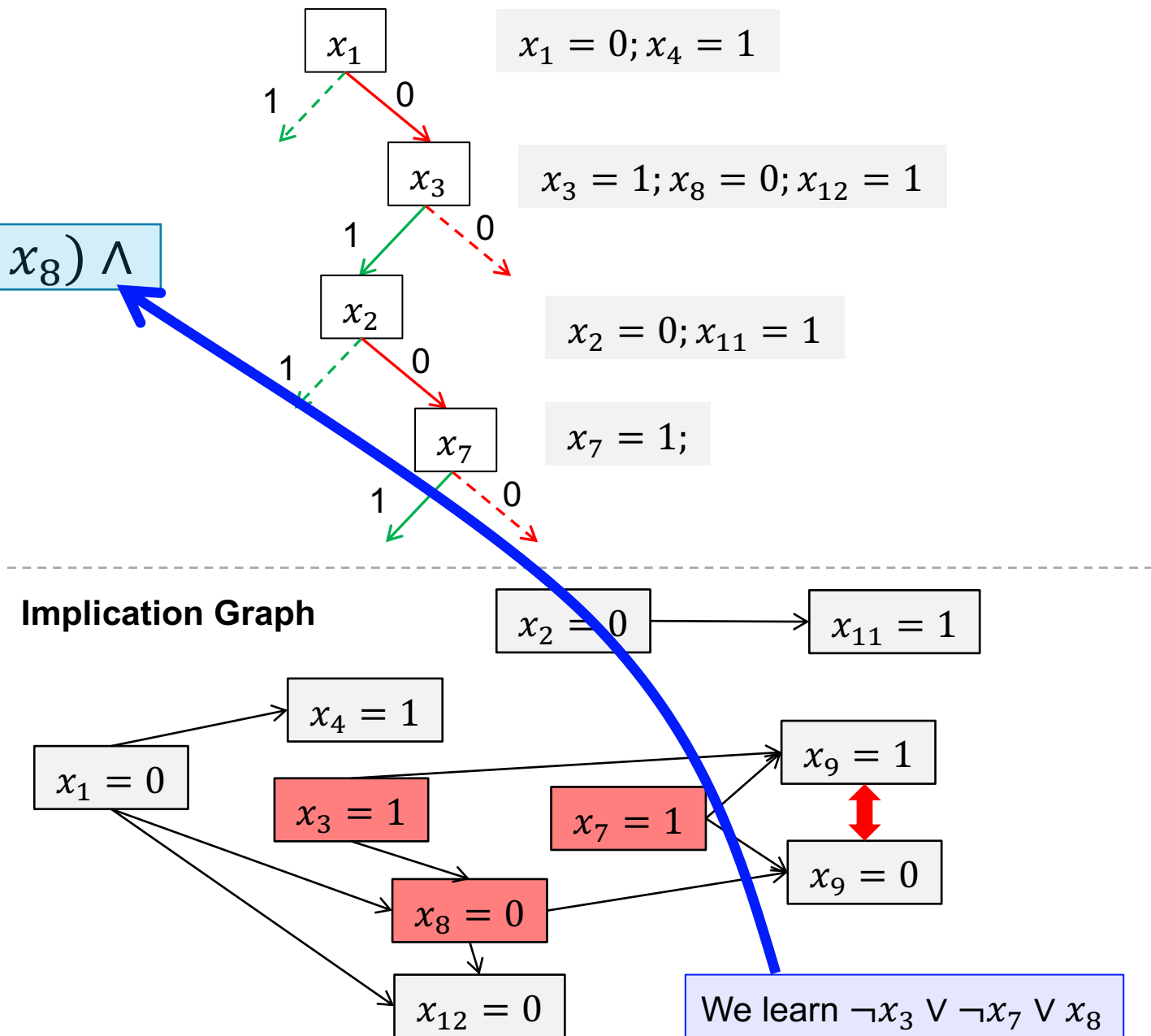
Implication Graph



We learn $\neg x_3 \vee \neg x_7 \vee x_8$

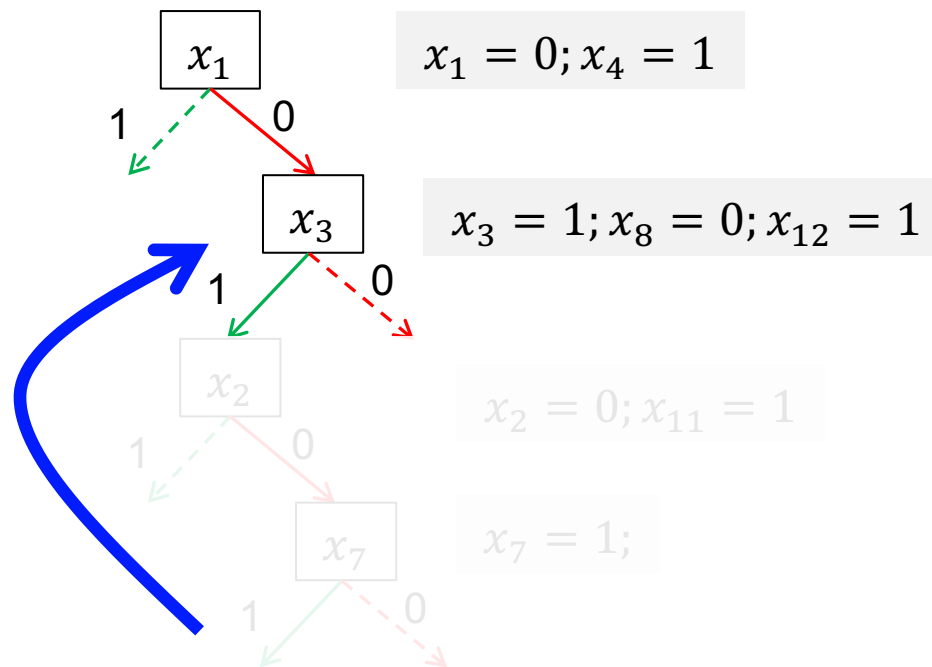
CDCL

- $(x_1 \vee x_4) \wedge (\neg x_3 \vee \neg x_7 \vee x_8) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

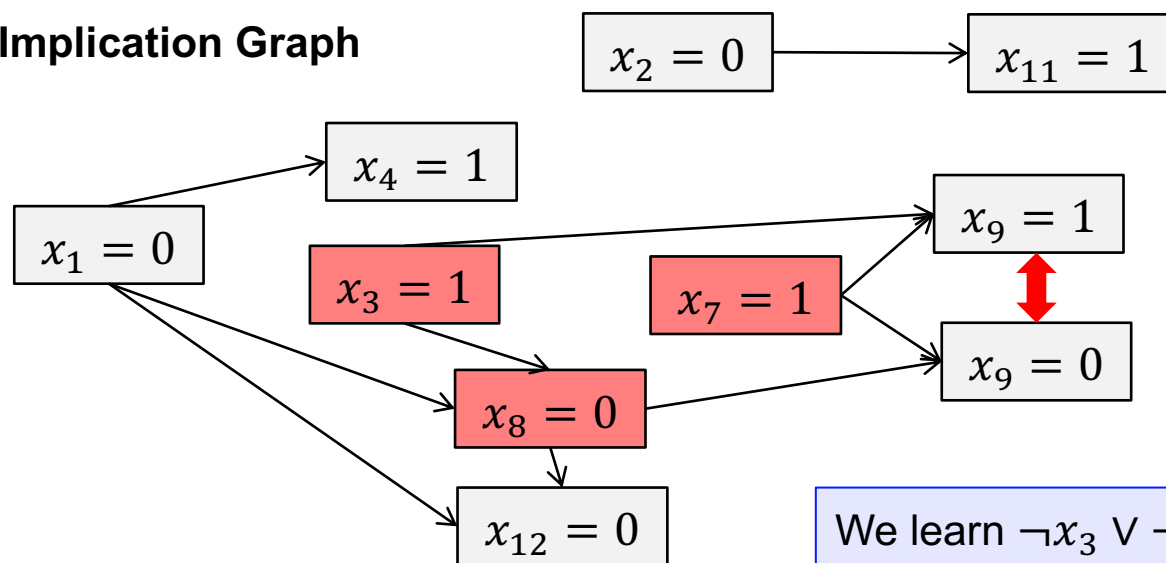


CDCL

- $(x_1 \vee x_4) \wedge (\neg x_3 \vee \neg x_7 \vee x_8) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$



Implication Graph



We learn $\neg x_3 \vee \neg x_7 \vee x_8$

Recap

**Given a formula F in propositional logic,
DPLL/CDCL can be used to decide its satisfiability!**

First-Order Logic

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by
 - (1) Symbols for objects, e.g., $a, b, c, \dots$
 - (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$
 - (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$
 - (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$
- } Terms (t)
- A first-order language is often used with a “**theory**” that defines symbols and relationships

Theory of Bit Vectors

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by
 - (1) Symbols for objects, e.g., $a, b, c, \dots$
 - (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$
 - (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$
 - (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

Each symbol is a bit vector of a fixed width.

- A first-order language is often used with a “**theory**” that defines symbols and relationships

Theory of Bit Vectors

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by

- (1) Symbols for objects, e.g., $a, b, c, \dots$

- (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$

- (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$

- (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

Relational/Logic operations between bitvectors, e.g., s/t , ult , $\wedge$, $\vee \dots$

- A first-order language is often used with a “**theory**” that defines symbols and relationships

Theory of Bit Vectors

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by

- (1) Symbols for objects, e.g., $a, b, c, \dots$
- (2) N-ary Predicates (relations returning true/false), e.g., $P(a_1, a_2, \dots, a_n)$
- (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$

Arithmetic operations, e.g.,
 $+, -, *, /, \%$, etc.

Bit vector's "+" is different. $\therefore$

- (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

- A first-order language is often used with a **"theory"** that defines symbols and relationships

Theory of Bit Vectors

- How can we check the satisfiability of a constraint?

```

1. void foobar(int a, int b) {
2.     int x = 1, y = 0;
3.     if (a != 0) {
4.         y = 3+x;
5.         if (b == 0)
6.             x = 2*(a+b);
7.     }
8.     assert(x-y != 0);
9. }

```

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

32-bit bitvector for the type int

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 1:

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0) \xrightarrow{\text{Step 1}} \alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \alpha_a = 2$$

$$\xrightarrow{\text{Step 1}} \alpha_b = 0 \wedge \alpha_a = 2$$

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 2:

$x = [a_7, a_6, \dots, a_0]$, $y = [b_7, b_6, \dots, b_0]$ and $z = [c_7, c_6, \dots, c_0]$ are three 8bit bit vectors

$$x \mid y = z \xrightarrow{\text{Step 2}} \bigwedge_{i=0}^7 ((a_i \vee b_i) \Leftrightarrow c_i)$$

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 2:

$x = [a_7, a_6, \dots, a_0]$, $y = [b_7, b_6, \dots, b_0]$ and $z = [c_7, c_6, \dots, c_0]$ are three 8bit bit vectors

$$x >_u 0 \xrightarrow{\text{Step 2}} \bigvee_{i=0}^7 a_i \qquad x >_s 0 \xrightarrow{\text{Step 2}} \neg a_7 \wedge \bigvee_{i=0}^6 a_i$$

Summary

